

Usage of an API for an integrator

Contents

1	Input from Product Manager	2
2	Authentication	2
2.1	Get the values from the environment variables (e.g. .env file)	2
2.2	Get an access token from Entra	2
3	Use the endpoint	3
3.1	Execute the call against the endpoint	3
3.2	Use the reponse	3
4	Notes	4
4.1	Secrets Management	4
5	Secrets Expiration	4

The following describes on how to use an API when the following parameters have been given to an integrator:

- **TENANT_ID**: Is always the same for the city of Biel/Bienne
- **CLIENT_ID**: Id of the Client App that has the appropriate permissions to use an API
- **SCOPE**: Scope/Audience to use when authenticating and requesting a token. The scope has the form `api://{api-id}/.default`
- **SECRET**: Client Secret. **Keep it secure, do not commit to any repository!**

The URLs and documentation of the APIs are available at [<https://apim-biel-bienne.developer.azure-api.net/>].

Code is illustrated as simple HTTP requests in Python and can easily be translated into any language. It represents the least common denominator. Feel free to use higher leven

authentication libraries such as `msal`.

1 Input from Product Manager

```
=====
SETUP COMPLETED SUCCESSFULLY
=====
Tenant ID:          c8915bd6-40a5-48e2-9a1e-d0dca8070588
API App Name:       graph-entra-reader-api-prod
API App ID:         f1f9dc21-d03e-4b58-b769-fee4724b8b5c
API Service Princ:  c1d5ae2f-48a9-40d8-a2e6-0e6698cc4188
Client App Name:    digitale-identitaet-data-analytics-prod
Client App ID:      562aa0d2-fe4e-447a-b250-21a39acfb04
Client Svc Princ:   03432630-14e7-4aca-a57f-a4480c6dab93

=== NEW CLIENT SECRET DETAILS ===
Secret Name:        digitale-identitaet-data-analytics-prod-Secret
    ↪ -202607031214
Valid Until:        07/03/2028 12:14:34
Secret Value:       abc1D~56oW2DcQr4abAsWelostM0QMmVtQsRcwoc
=====
```

You'll need: * Tenant ID * API App ID * Client App ID * Secret Value

2 Authentication

2.1 Get the values from the environment variables (e.g. `.env` file)

```
tenant_id = os.getenv("TENANT_ID")
client_id = os.getenv("CLIENT_ID")
client_secret = os.getenv("CLIENT_SECRET")
scope = os.getenv("SCOPE")
```

2.2 Get an access token from Entra

```
token_url = f"https://login.microsoftonline.com/{tenant_id}/oauth2/v2.0/"
    ↪ token"
token_data = {
    "client_id": client_id,
    "client_secret": client_secret,
    "scope": scope,
    "grant_type": "client_credentials"
}
```

```

token_response = requests.post(token_url, data=token_data)

if not token_response.ok:
    print("Fehler beim Abrufen des Tokens:", token_response.text)
    token_response.raise_for_status()

access_token = token_response.json()["access_token"]

```

3 Use the endpoint

3.1 Execute the call against the endpoint

```

apim_endpoint = "https://apim-biel-bienne.azure-api.net/digitale-
    ↪ identitaet/graph-entra-reader/users"

# Query parameter für Graph API ($top=10)
params = {
    "$top": 10
}

headers = {
    "Authorization": f"Bearer {access_token}",
    "Accept": "application/json"
}

api_response = requests.get(apim_endpoint, headers=headers, params=params
    ↪ )

if not api_response.ok:
    print("Error calling API:", api_response.text)
    api_response.raise_for_status()

users_data = api_response.json()

```

3.2 Use the reponse

```

for user in users_data.get("value", []):
    print(f"- {user.get('displayName')} ({user.get('userPrincipalName')})
    ↪ ")

```

4 Notes

- Do **not** use the interactive authentication from the developer portal
- A client app may have access to several APIs, but will need to authenticate with different scopes
- Keep the secret secure. Typical approaches include:
 - KeyVault, e.g. Azure KeyVault.
 - Environment variable of the runtime application (injected)
 - .env file with the environment variables (useful during development). **Add to .gitignore**

4.1 Secrets Management

KeyVaults are the preferred way to productively store a secret. For example, in an Azure environment, only the system assigned managed identity of the Web App (or Function App) has a role to read secrets. Not even administrators can read secrets. Upon startup, the KeyVault secrets are read using a Managed Identity authentication and are then available in memory during runtime.

Alternatives to Azure KeyVault are:

- AWS Secrets Manager
- Google Cloud Secret Manager
- On Premises: HashiCorp Vault

5 Secrets Expiration

Secrets have a validity and will expire. The duration is dependent on the confidentiality of the data accessed and ranges from 6 months to 24 months.

You'll be notified when the secret needs to be rotated.